

DPDP COMPLIANCE AUDIT

Agrivista

Digital Personal Data Protection Act 2023



Grade C — Needs Work

Scanned: 13 April 2026

Files indexed: 17

Languages: express, mongoose, passport, passport-local-mongoose, ejs, ejs-mate

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

Quick Actions for Developers

Fix these first to improve your compliance score:

- **CONSENT_WITHDRAWAL_MISSING** (codebase-wide)
- **AUDIT_TRAIL_MISSING** (codebase-wide)
- **NO_DELETION_MECHANISM** → src/app.js
- **DATA_ACCESS_MISSING** (codebase-wide)
- **RETENTION_MISSING** → src/models/user.model.js

This codebase has 5 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 5 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 3.2–11.8 engineering days.

The application collects user email, username, and password for authentication, along with detailed agricultural profile data. The biggest DPDP risk is the collection and transmission of agricultural input data (N, P, K, temperature, humidity, ph, rainfall, fertilizer, pesticide) to external ML APIs without explicit mention of consent or purpose limitation for this data.

Total findings: 17

- Rule Engine: 12
- AI Deep Review: 5

By Severity:

- HIGH: 5
- MEDIUM: 5
- LOW: 3
- INFO: 4

Developer Priority:

- Fix Now: 5
- Fix This Sprint: 3

Tech Stack: express, mongoose, passport, passport-local-mongoose, ejs

Risk Profile: The application collects user email, username, and password for authentication, along with detailed agricultural profile data. The biggest DPDP risk is the collection and transmission of agricultural input data (N, P, K, temperature, humidity, ph, rainfall, fertilizer, pesticide) to external ML APIs without explicit mention of consent or purpose limitation for this data.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	22/22 (100%)	PASS
Section 6(6)	8	0/8 (0%)	FAIL
Section 7	4	4/4 (100%)	PASS
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	0/10 (0%)	FAIL
Section 8(3)	8	4.0/8 (50%)	WARN
Section 8(4)	8	0/8 (0%)	FAIL

Section 8(6)	8	6.8/8 (85%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	2.1/6 (35%)	FAIL
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 3.2–11.8 engineering days

Scan Coverage

Files analyzed: 17

Rule	Section	Severity	File
CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
NO_DELETION_MECHANISM	Section 8 — Data Principal Rights	HIGH	src/app.js
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
DATA_ACCESS_MISSING	Section 11 — Right to Access	MEDIUM	N/A
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	src/models/user.model.js
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
INTERNAL_ROUTE_PII_ACCESS	Section 7 — Legitimate Use	INFO	src/app.js
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
DEEP_REVIEW_VALIDATED	Article 5(1)(c) — Data Minimisation	MEDIUM	src/controllers/user.controller.js
DEEP_REVIEW_VALIDATED	Article 5(1)(a) — Lawfulness, Fairness and Transparency	LOW	src/middleware/isLoggedIn.middleware.js
DEEP_REVIEW_VALIDATED	Article 5(1)(b) — Purpose Limitation	MEDIUM	src/models/profile.model.js
DEEP_REVIEW_VALIDATED	Article 5(1)(b) — Purpose Limitation	MEDIUM	src/models/user.model.js

DEEP_REVIEW_VALIDATED	Article 5(1)(a) — Lawfulness, Fairness and Transparency, Article 5(1)(b) — Purpose Limitation, Article 5(1)(d) — Data Minimisation	HIGH	src/controllers/CropPrediction.controllers.js
-----------------------	--	------	---

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (10–34 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Withdrawal Missing (1–4 hrs)
- Audit Trail Missing (4–12 hrs)
- No Deletion Mechanism — src/app.js (2–6 hrs)
- Grievance Officer Missing (1–4 hrs)
- Deep Review Validated — controllers/CropPrediction.controllers.js (2–8 hrs)

Week 2 — High Priority (10–38 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Data Access Missing (2–6 hrs)
- Retention Missing — models/user.model.js (2–8 hrs)
- Deep Review Validated — controllers/user.controller.js (2–8 hrs)
- Deep Review Validated — models/profile.model.js (2–8 hrs)
- Deep Review Validated — models/user.model.js (2–8 hrs)

Week 4 — Advisory (6–22 hours)

Address before next compliance review.

- No Breach Alerting (2–6 hrs)
- Data Portability Missing (2–8 hrs)
- Deep Review Validated — middleware/isloggedin.middleware.js (2–8 hrs)

Total estimated remediation effort: 26–94 hours (3.2–11.8 engineering days)

Details

Finding 1 of 17 — ■ FIX NOW

[Section 6(6) — Consent Withdrawal]

1. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A:15**Evidence snippet:** The signup controller handles user registration, collecting username, email, and password.**Estimated fix time:** 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

The application collects user consent for processing personal data, including sensitive agricultural information, but lacks a mechanism for users to withdraw this consent. This violates DPDP Section 6(6), potentially leading to continued processing of data against the user's wishes and legal non-compliance.

Fix:

1. Create a new route, e.g., '/consent/withdraw', that a logged-in user can access.
2. In the corresponding controller, implement logic to mark user consent as withdrawn for all data processing activities.
3. Ensure this withdrawal mechanism is as easy to access as the consent giving process, potentially by adding a link in the user's profile or settings page.
4. Update the application's data handling logic to respect withdrawn consent and cease processing personal data accordingly.

DPDP:

Section 6(6) — Consent Withdrawal requires that consent withdrawal must be as easy as giving consent.

Example:

```
router.get('/profile/consent', (req, res) => { res.render('consent_withdrawal', { userId: req.user._id }); }); router.post('/profile/consent/withdraw', async (req, res) => { try { await User.findByIdAndUpdate(req.user._id, { consentGiven: false }); req.flash('success', 'Your consent has been withdrawn.');
```

```
res.redirect('/profile'); } catch (err) { req.flash('error', 'Failed to withdraw consent.');
```

```
res.redirect('/profile'); } });
```

Finding 2 of 17 — ■ FIX NOW

[Section 8(4) — Audit Trail]

2. [HIGH] AUDIT_TRAIL_MISSING — N / A:100**Evidence snippet:** The application sets up session management using express-session.**Estimated fix time:** 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The application lacks a comprehensive audit trail for personal data processing activities, failing to meet DPDP Section 8(4) requirements. Without logging access, modifications, or deletions of personal data, it's impossible to ensure accountability or respond effectively to data subject requests, increasing the risk of unauthorized access and misuse.

Fix:

1. Create a new Mongoose model, e.g., `AuditLog`, to store audit records.
2. Implement middleware that logs key events such as user login/logout, profile updates, and data access, including user ID, timestamp, action, and affected data.
3. Integrate this middleware into the application's request lifecycle, ensuring it captures relevant operations.
4. Add a route to view audit logs, accessible only to authorized administrators.

DPDP:

Section 8(4) — Audit Trail requires complete records of personal data processing activities.

Example:

```
const auditLogSchema = new mongoose.Schema({ userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }, action: String, timestamp: { type: Date, default: Date.now }, details: Object }); const AuditLog = mongoose.model('AuditLog', auditLogSchema); app.use(async (req, res, next) => { await AuditLog.create({ userId: req.user ? req.user._id : null, action: `${req.method} ${req.originalUrl}`, details: { ip: req.ip, body: req.body } }); next(); });
```

Finding 3 of 17 — ■ FIX NOW

[Section 8 — Data Principal Rights]

3. [HIGH] NO_DELETION_MECHANISM — src / app.js:100**Evidence snippet:** The application uses express-session for managing user sessions.**Estimated fix time:** 2–6 hours

No data deletion or account erasure mechanism detected. Users may not be able to exercise their right to erasure under DPDP Section 8.

Risk:

The application does not provide a mechanism for users to request the deletion of their personal data or account, which is a violation of DPDP Section 8. This prevents users from exercising their right to erasure, potentially leading to non-compliance and user dissatisfaction.

Fix:

1. Create a new route, e.g., '/profile/delete', that a logged-in user can access.
2. Implement a controller function for this route that first prompts the user for confirmation.
3. Upon confirmation, add logic to delete the user's `Profile` data and then the user's `User` record.
4. Ensure all associated data, including profile information, is removed from the system.

DPDP:

Section 8 — Data Principal Rights includes the right to erasure of personal data.

Example:

```
router.post('/profile/delete', async (req, res) => { try { await Profile.deleteOne({ owner: req.user._id }); await User.findByIdAndDelete(req.user._id); req.logout(); // Log the user out after deletion req.flash('success', 'Your account and profile have been deleted.')} catch (err) { req.flash('error', 'Failed to delete account.')} res.redirect('/'); } });
```

Finding 4 of 17 — ■ FIX NOW

[Section 13 — Grievance Redressal]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A:121

Evidence snippet: The middleware ``res.locals.currentUser = req.user;`` makes the current user object available in rendered templates.

Estimated fix time: 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route ****and**** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The application does not designate or publish contact information for a Grievance Officer or Data Protection Officer as required by DPDP Section 13. This absence makes it difficult for data principals to lodge complaints or seek redressal regarding their personal data, hindering their rights and potentially leading to non-compliance.

Fix:

1. Create a new route, e.g., `/grievance`, that provides information on how to contact the grievance officer.
2. In the corresponding controller, render an EJS template that includes the grievance officer's contact details (e.g., email address).
3. Add a clear link to this grievance route in the application's footer or user settings page.
4. Ensure the contact email provided is actively monitored for grievance-related inquiries.

DPDP:

Section 13 — Grievance Redressal requires the appointment and publication of a Grievance Officer.

Example:

```
router.get('/grievance', (req, res) => { res.render('grievance', { grievanceOfficerEmail:
process.env.GRIEVANCE_OFFICER_EMAIL || 'dpo@example.com' }); }); // In your footer or settings
view: // <a href="/grievance">Grievance Redressal</a>
```


Finding 5 of 17 — ■ REVIEW & FIX

[Section 11 — Right to Access]

5. [MEDIUM] DATA_ACCESS_MISSING — N / A**Estimated fix time:** 2–6 hours

No endpoint detected that allows authenticated users to retrieve their own personal data. Under DPDP Section 11, Data Principals have the right to access information about their personal data being processed.

Risk:

The application lacks an endpoint for authenticated users to retrieve their personal data, violating the Data Principal's right to access under DPDP Section 11. This prevents users from understanding what data is held about them.

Fix:

1. Create a new GET route, e.g., '/my-data', accessible only to authenticated users.
2. In the new route handler, fetch the current user's data from the database using `req.user`.
3. Include relevant personal data (e.g., email, profile details) in the response, formatted as JSON.
4. Ensure the response is sent back to the client, e.g., `res.json(userData)`.

DPDP:

Section 11 — Right to Access — Data Principals have the right to access information about their personal data being processed.

Example:

```
app.get('/my-data', (req, res) => { if (!req.user) { return res.status(401).json({ message: 'Unauthorized' }); } // Fetch additional profile data if needed // const profileData = await Profile.findOne({ owner: req.user._id }); res.json({ userId: req.user._id, email: req.user.email, // Add other relevant fields from user and profile models }); });
```

Finding 6 of 17 — ■ REVIEW & FIX

[Section 8(3) — Retention]

6. [MEDIUM] RETENTION_MISSING — src / models / user.model.js**Estimated fix time:** 2–8 hours

Model/schema file contains PII fields but no retention or expiry signals detected. Data may be retained indefinitely.

Risk:

The user and profile schemas contain personal data fields without any defined retention periods or expiry mechanisms. This could lead to indefinite storage of personal data, violating DPDP Section 8(3) requirements for data minimization and purpose limitation.

Fix:

1. Add a `dataRetentionPeriod` field (e.g., in days or months) to your Mongoose schemas (e.g., `userSchema` and `profileSchema`).
2. Implement a scheduled job or a pre-save hook to check and enforce data retention policies.
3. Consider adding an `expiresAt` field that is automatically set based on the retention period and use MongoDB's TTL index for automatic deletion.
4. Document the data retention policy in your privacy policy and internal data handling procedures.

DPDP:

Section 8(3) — Retention — Personal data shall not be retained for longer than is necessary for the purpose for which it is processed.

Example:

```
const userSchema = new Schema({ email: { type: String, required: true }, createdAt: { type: Date, default: Date.now }, dataRetentionDays: { type: Number, default: 365 } // Example: 1 year retention }); userSchema.index({ createdAt: 1 }, { expireAfterSeconds: 30 * 24 * 60 * 60 }); // Example: 30 days // Or, for profileSchema: const profileSchema = new Schema({ // ... other fields owner: { type: Schema.Types.ObjectId, ref: "User" }, createdAt: { type: Date, default: Date.now }, dataRetentionDays: { type: Number, default: 730 } // Example: 2 year retention }); profileSchema.index({ createdAt: 1 }, { expireAfterSeconds: 60 * 24 * 60 * 60 }); // Example: 60 days
```

Finding 7 of 17 — ■ BACKLOG

[Section 8(6) — Breach]

7. [LOW] NO_BREACH_ALERTING — N / A**Estimated fix time:** 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

While the application has some logging (e.g., console logs for process events and database connections), there is no structured mechanism for alerting on security breaches. This means that if a data breach occurs, affected individuals and the Authority may not be notified promptly as required by DPDP Section 8(6).

Fix:

1. Integrate a dedicated error tracking and alerting service like Sentry or LogRocket.
2. Configure the service to monitor application errors and security-related events.
3. Set up alerts to notify a designated compliance team or individual via email or Slack when a critical event (potential breach) is detected.
4. Ensure that PII field access or modification attempts are logged and trigger alerts.

DPDP:

Section 8(6) — Breach — Data Fiduciaries shall without undue delay, notify the Authority and the affected Data Principal of any breach of personal data.

Example:

```
// Example using Sentry (install @sentry/node) import * as Sentry from '@sentry/node';
Sentry.init({ dsn: process.env.SENTRY_DSN, integrations: [ new Sentry.Integrations.Express({
  app }), ], tracesSampleRate: 1.0, }); // In your error handling middleware or specific routes:
// Sentry.captureException(error); // Sentry.captureMessage('Potential data breach
detected!');
```

Finding 8 of 17 — ■ BACKLOG

[Section 11 — Data Portability]

8. [LOW] DATA_PORTABILITY_MISSING — N / A**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The application does not offer any endpoint for users to export their personal data in a machine-readable format. This contravenes DPDP Section 11, which mandates that Data Fiduciaries provide personal data upon request in a commonly used, interoperable format.

Fix:

1. Create a new GET route, e.g., '/export-data', accessible to authenticated users.
2. In the route handler, fetch the user's personal data (e.g., from User and Profile models).
3. Format the data into a machine-readable format like JSON or CSV.
4. Set appropriate response headers (e.g., 'Content-Disposition: attachment; filename="user_data.json"') and send the data as a file download.

DPDP:

Section 11 — Data Portability — Data Fiduciaries shall provide the personal data of the Data Principal in a structured, commonly used and machine-readable format.

Example:

```
app.get('/export-data', async (req, res) => { if (!req.user) { return  
res.status(401).send('Unauthorized'); } try { const userData = await  
User.findById(req.user._id).lean(); const profileData = await Profile.findOne({ owner:  
req.user._id }).lean(); const dataToExport = { ...userData, profile: profileData };  
res.setHeader('Content-Type', 'application/json'); res.setHeader('Content-Disposition',  
'attachment; filename="user_data.json"'); res.json(dataToExport); } catch (error) {  
res.status(500).send('Error exporting data'); } }));
```

Finding 9 of 17 — ■ BACKLOG

[Section 7 — Legitimate Use]

9. [INFO] INTERNAL_ROUTE_PII_ACCESS — src / app.js

Internal file (app.js) accesses PII. Verify access is restricted to authorised contexts and PII is not exposed to logs or error responses.

Finding 10 of 17 — ■ BACKLOG

[Section 5 — Notice]

10. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 11 of 17 — ■ BACKLOG

[Section 5 — Purpose Limitation]

11. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 12 of 17 — ■ BACKLOG

[Section 16 — Cross-Border Transfer]

12. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 13 of 17 — ■ FIX THIS SPRINT

[Article 5(1)(c) — Data Minimisation]

13. [MEDIUM] DEEP_REVIEW_VALIDATED [AI Deep Review] — src / controllers / user.controller.js**Estimated fix time:** 2–8 hours

The `catch` block in `saveSignup` directly flashes the error message `e.message` to the user. This message could potentially contain sensitive PII if the error originates from a PII-related field.

Finding 14 of 17 — ■ BACKLOG

[Article 5(1)(a) — Lawfulness, Fairness and Transparency]

14. [LOW] DEEP_REVIEW_VALIDATED [AI Deep Review] — src / middleware / isloggedin.middleware.js**Estimated fix time:** 2–8 hours

The `isLoggedIn` middleware stores the original URL in `req.session.redirectUrl` when a user is not authenticated.

Finding 15 of 17 — ■ FIX THIS SPRINT

[Article 5(1)(b) — Purpose Limitation]

15. [MEDIUM] DEEP_REVIEW_VALIDATED [AI Deep Review] — src / models / profile.model.js**Estimated fix time:** 2–8 hours

The `profileSchema` includes fields like `number`, `State`, and `city` which can be considered personal information. These fields are stored without any explicit indication of the purpose for which they are collected or processed.

Finding 16 of 17 — ■ FIX THIS SPRINT

[Article 5(1)(b) — Purpose Limitation]

16. [MEDIUM] DEEP_REVIEW_VALIDATED [AI Deep Review] — src / models / user.model.js**Estimated fix time:** 2–8 hours

The `userSchema` includes an `email` field which is marked as required. However, there is no explicit field or metadata indicating the specific, legitimate purpose for collecting and storing this email address.

Finding 17 of 17 — ■ FIX NOW

[Article 5(1)(a) — Lawfulness, Fairness and Transparency, Article 5(1)(b) — Purpose Limitation, Article 5(1)(d) — Data Minimisation]

17. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — src / controllers / CropPrediction.controllers.js**Estimated fix time:** 2–8 hours

The `resultCropPrediction` function sends user-provided crop data (N, P, K, temperature, humidity, ph, rainfall) to an external API (`https://agrivista-ml-api.onrender.com/predict-crop`) without any apparent consent mechanism or data minimization check.

AI Deep Compliance Review

This section shows AI analysis that identified compliance gaps beyond deterministic rules. Validated findings are integrated into the main findings list above with [AI Deep Review] labels.

Overall Risk: CRITICAL

The application's handling of user data, particularly sensitive agricultural inputs, presents a critical risk to DPDP compliance. Personal information is collected and transmitted to external services without explicit user consent or clear purpose limitation. This lack of transparency and control over data flows exposes the organization to significant legal and reputational damage.

Estimated remediation: 15 engineering day(s)

Most Critical Action: Implement a clear consent mechanism before transmitting agricultural input data to the external ML API, detailing the purpose and scope of data usage.

- **Lack of explicit consent and purpose limitation for PII [HIGH]** — Article 5 (Principles relating to processing of personal data) — layers: Data Models & Storage, API Routes & Controllers, Third-Party & PII Flows

- **Inadequate error handling leading to potential PII leakage [MEDIUM]** — Article 32 (Security of processing) — layers: Third-Party & PII Flows, API Routes & Controllers

1. [HIGH] Sensitive data sent to external API without explicit consent — API Routes & Controllers — Article 6 (Lawfulness of processing)

User-provided crop data (N, P, K, temperature, humidity, ph, rainfall) is sent to an external API without any explicit consent mechanism.

2. [MEDIUM] PII stored without explicit purpose limitation — Data Models & Storage — Article 5 (Principles relating to processing of personal data)

Profile fields like 'number', 'State', and 'city' are stored without explicit indication of their purpose.

3. [MEDIUM] User email stored without explicit purpose — Data Models & Storage — Article 5 (Principles relating to processing of personal data)

The 'email' field is required but lacks explicit metadata on its specific, legitimate purpose.

4. [MEDIUM] PII exposed in error messages — Third-Party & PII Flows — Article 32 (Security of processing)

Error messages containing 'e.message' are directly flashed to users, potentially revealing sensitive PII.

5. [MEDIUM] Error handling swallows external API errors — API Routes & Controllers — Article 32 (Security of processing)

External API errors are logged to the console but not fully detailed to the user or logged comprehensively for auditing.

Layer Breakdown

Third-Party & PII Flows — 2 finding(s). The layer shows basic PII handling for signup and login. However, there are opportunities to improve error message sanitization and ensure consistent error handling to prevent potential data leakage and maintain control integrity.

Authentication & Session — 1 finding(s). The authentication middleware correctly identifies unauthenticated users and redirects them. However, it stores the original URL in the session, which could potentially include sensitive information, without explicit consideration for data protection principles.

Data Models & Storage — 2 finding(s). The data models store personal information (email, location data) without explicit purpose limitation, which is a potential DPDP compliance risk. The database connection itself is standard.

API Routes & Controllers — 3 finding(s). The API routes controller handles crop prediction, interacting with an external ML API. A key concern is the transmission of potentially sensitive data to this external service without clear consent, and the error handling could be more robust for auditing purposes. Commented-out code also reduces clarity.